

Memory Virtualization

- Address Space and Translation -

Youngjae Kim (Ph.D.)

Sogang University

Linux 운영체제 및 응용 (GITF315-01)

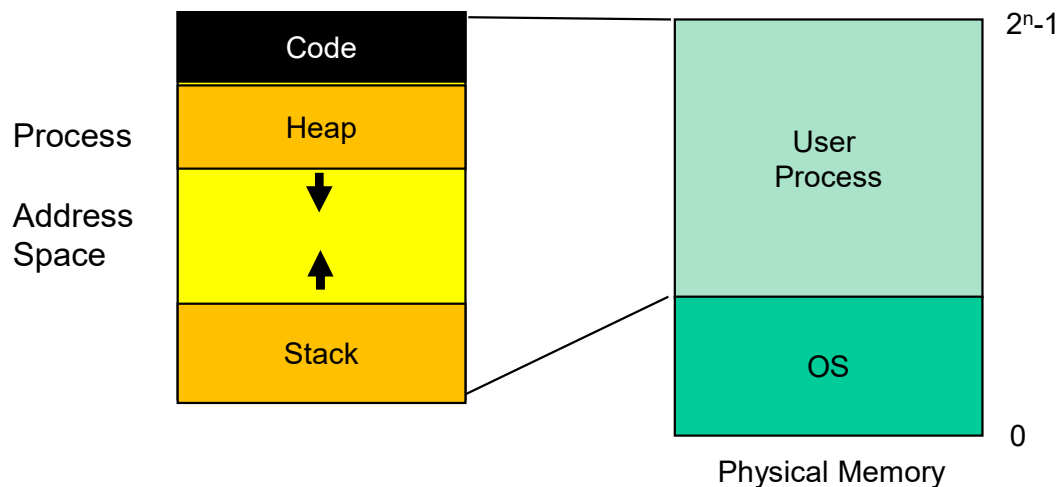
Virtual Memory

Memory Virtualization

■ What is memory virtualization?

- OS virtualizes its physical memory
- OS provides an **illusion memory space** per each process
 - Virtual RAM: *illusion* of **private memory**
- It seems to be seen **like each process uses the whole memory**

■ Uniprogramming: One process runs at a time



Disadvantages:

- Only one process runs at a time
- Process can destroy OS

Motivation for Dynamic Memory

■ Why do processes need dynamic allocation of memory?

- Do not know amount of memory needed at compile time
- Must be pessimistic when allocate memory statically
 - Allocate enough for worst possible case; Storage is used inefficiently

■ Recursive procedures

- Do not know how many times procedure will be nested

■ Complex data structures: lists and trees

- ```
struct my_t *p
 = (struct my_t *)malloc(sizeof(struct my_t));
```

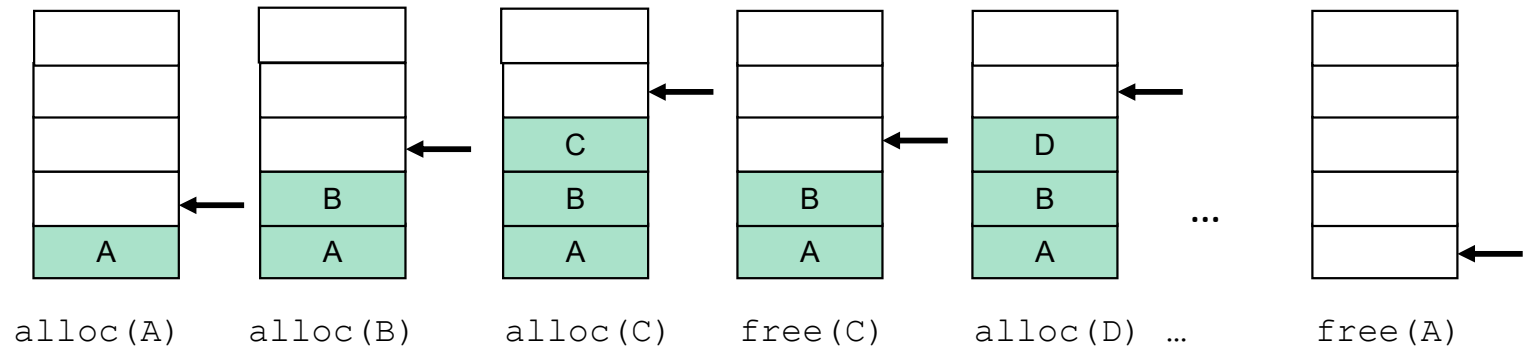
## ■ Two types of dynamic allocation

- Stack
- Heap

# Stack Organization

## ■ Definition: Memory is freed in opposite order from allocation

```
alloc(A);
alloc(B);
alloc(C);
free(C);
alloc(D);
free(D);
free(B);
free(A);
```



## ■ Simple and efficient implementation: Pointer separates allocated and freed space

- ✓ Allocate: Increment pointer
- ✓ Free: Decrement pointer

## ■ No fragmentation

# Where Are Stacks Used?

- OS uses stack for procedure call frames (local variables and parameters)

```
main () {
 int A = 0;
 foo (A);
 printf("A: %d\n", A);
}

void foo (int Z) {
 int A = 2;
 Z = 5;
 printf("A: %d Z: %d\n", A, Z);
}
```

# Multiprogramming Goals

- **Transparency** 투명
  - Processes are not aware that memory is shared
  - Works regardless of number and/or location of processes
- **Protection** 보호
  - Cannot corrupt OS or other processes
  - Privacy: Cannot read data of other processes
- **Efficiency** 효율
  - Do not waste memory resources (minimize fragmentation)
- **Sharing** 공유
  - Cooperating processes can share portions of address space

# Abstraction: Address Space

- **Address space:** Each process has set of addresses that map to bytes
- **Address space has static and dynamic components**
  - Static: Code and some global variables
  - Dynamic: Stack and Heap
- **Problem:**
  - How can OS provide illusion of private address space to each process?



# Quiz: Match that Address Location

```

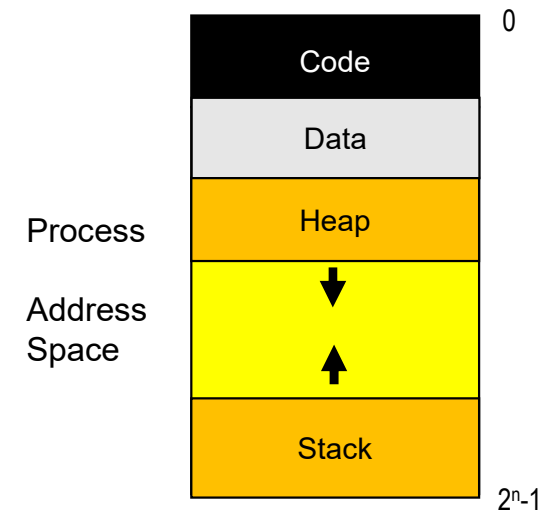
int x;
int main(int argc, char *argv[]) {
 int y;
 int *z = malloc(sizeof(int));
}

```

*Handwritten annotations:*  
 - **Code** (red) with an arrow pointing to `int x;`  
 - **Stack** (red) with an arrow pointing to `int y;`  
 - **Heap** (red) with an arrow pointing to `malloc(sizeof(int));`

Possible segments: static data, code, stack, heap

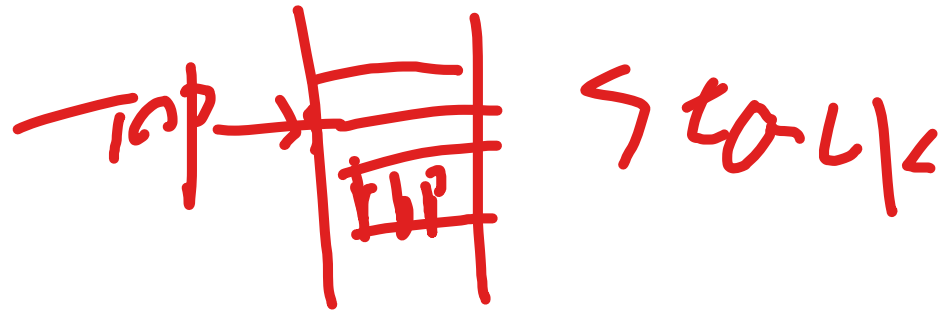
| Address | Location           |
|---------|--------------------|
| x       | Static data → Code |
| main    | Code               |
| y       | Stack              |
| z       | Stack              |
| *z      | Heap               |



*Handwritten note:* \*전역 변수는 Code, 지역 변수는 Stack, malloc으로 메모리를 할당하면 Heap

What if no static data segment?

# Memory Access



```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(int argc, char *argv[]) {
 int x;
 x = x + 3;
}
```

1. 이동한다 rbp보다 8 위로 가서 edi에 가져온다.
2. Edi 값에 3을 더한다.
3. Edi 값을 다시 rbp8에 넣는다.



```
0x10: movl 0x8(%rbp), %edi
0x13: addl $0x3, %edi
0x19: movl %edi, 0x8(%rbp)
```

otool -tv demo1.o  
(or objdump on Linux)

**%rbp** is the base pointer:  
points to base of **current stack frame**

# Quiz: Memory Accesses?

0x10: movl 0x8(%rbp), %edi

0x13: addl \$0x3, %edi

0x19: movl %edi, 0x8(%rbp)

➡ Initial %rip = 0x10  
%rbp = 0x200

**%rip** is instruction pointer (or program counter)

**%rbp** is the base pointer:  
points to base of current stack frame

**Memory Accesses to what addresses?**

Fetch instruction at addr 0x10

Exec:

load from addr 0x208

Fetch instruction at addr 0x13

Exec:

no memory access

Fetch instruction at addr 0x19

Exec:

store to addr 0x208

# Memory Virtualization

---

# How to Virtualize Memory?

- **Problem: How to run multiple processes simultaneously?**

Addresses are “hardcoded” into process binaries

- **How to avoid collisions?**
- **Possible Solutions for Mechanisms:**
  - 1) Time Sharing
  - 2) Static Relocation
  - 3) Base
  - 4) Base+Bounds

# 1) Time Sharing of Memory

- Try similar approach to how OS virtualizes CPU

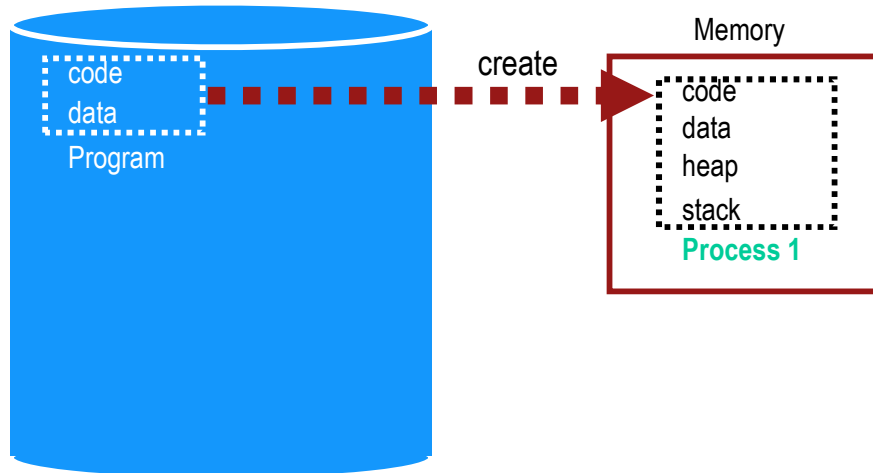
- Observation:

\*시간으로 나눠서 Memory를 차지함.

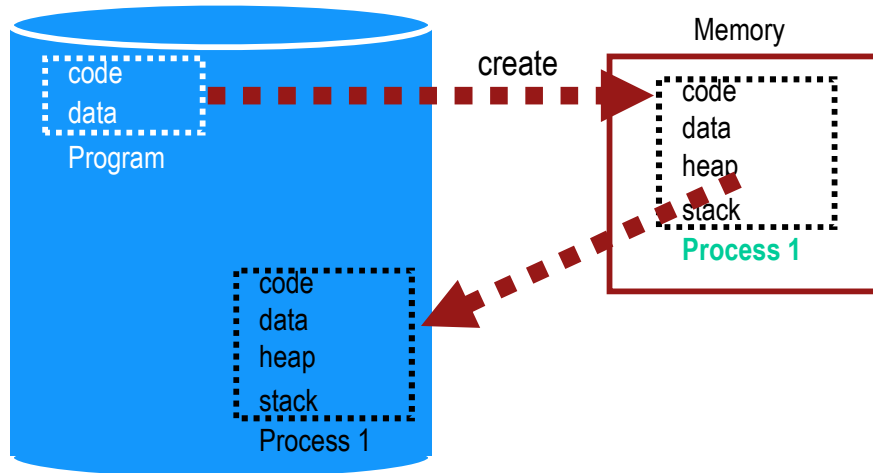
-> 한 타임에 하나의 process만 사용할 수 있어 매우 비효율적임.

- OS gives illusion of many virtual CPUs by saving CPU registers to memory when a process isn't running
- Could give illusion of many virtual memories by saving memory to disk when process isn't running

# 1) Time Sharing of Memory

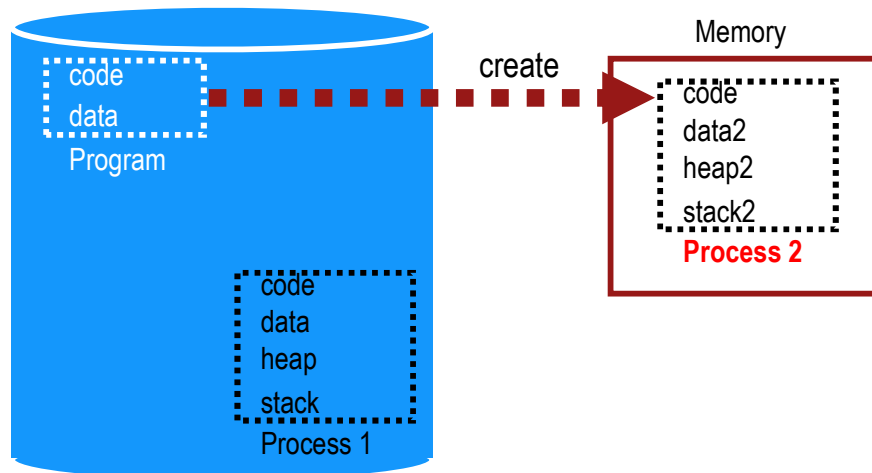
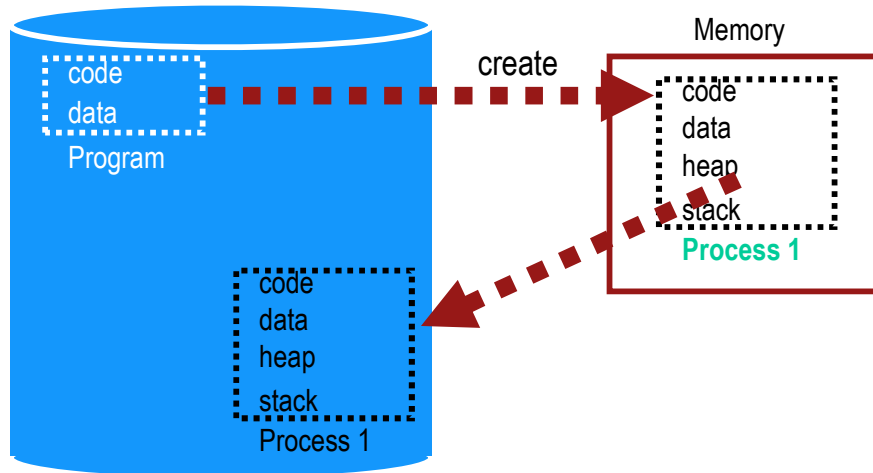


# 1) Time Sharing of Memory

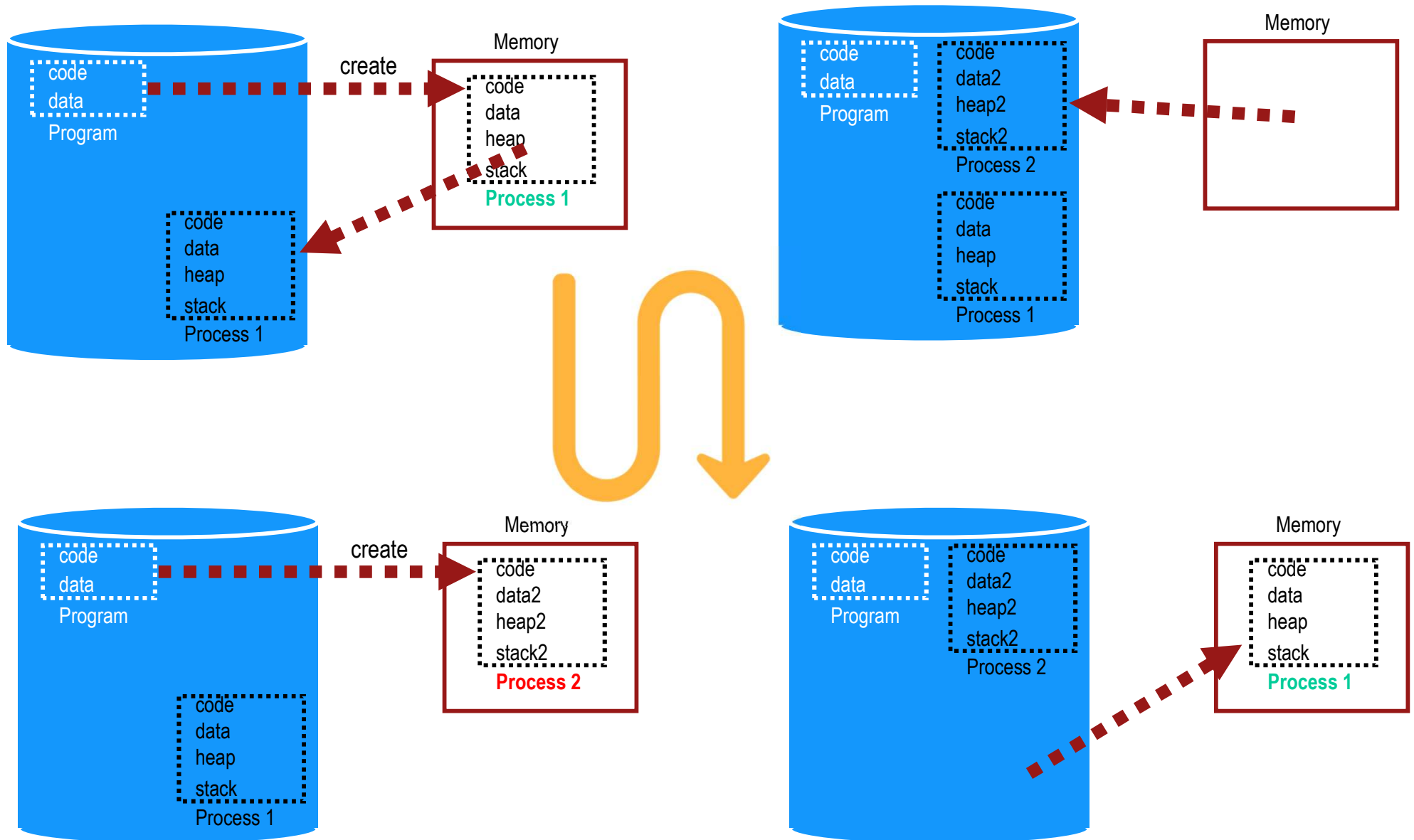




# 1) Time Sharing of Memory



# 1) Time Sharing of Memory



# Problems with Time Sharing Memory

- **Problem: Ridiculously poor performance**
- **Better Alternative: space sharing**
  - At same time, space of memory is divided across processes
- **Remainder of solutions all use space sharing**

## 2) Static Relocation

\*지역 별로 나누어 Memory를 할당함.

-> process가 memory에 올라갈 때 지역 번호를 붙여서 다시 할당(rewrite) 해야 하기 때문에 비효율적

- Idea: OS rewrites each program before loading it as a process in memory
- Each rewrite for different process uses different addresses and pointers
- Change jumps, loads of static data

rewrite

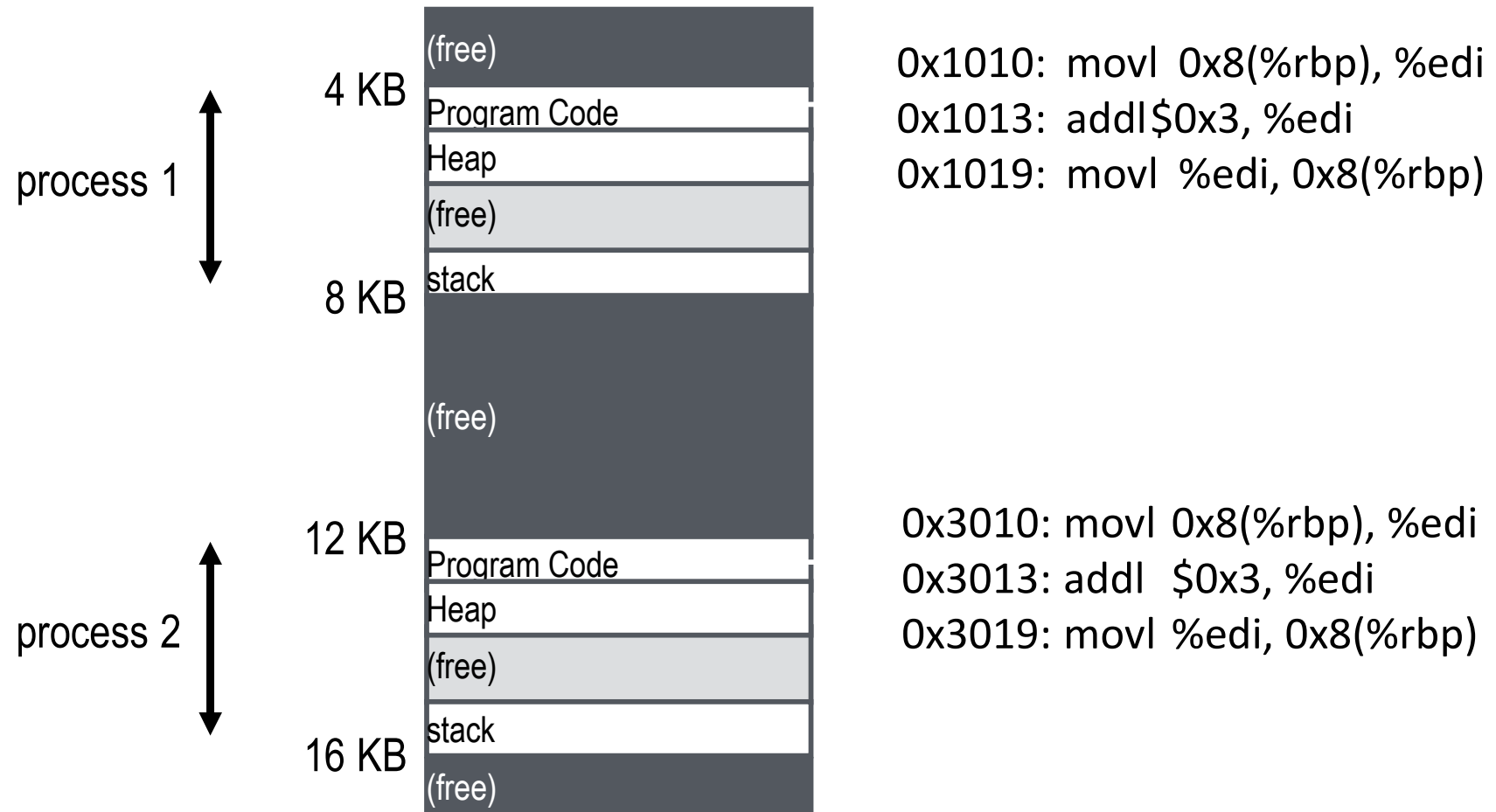
0x10 . .  
0x1010: movl 0x8(%rbp), %edi  
0x1013: addl \$0x3, %edi  
0x1019: movl %edi, 0x8(%rbp)

- 0x10: movl 0x8(%rbp), %edi
- 0x13: addl \$0x3, %edi
- 0x19: movl %edi, 0x8(%rbp)

rewrite

0x30 . .  
0x3010: movl 0x8(%rbp), %edi  
0x3013: addl \$0x3, %edi  
0x3019: movl %edi, 0x8(%rbp)

# Static: Layout in Memory



# Static Relocation: Disadvantages

## ■ No protection

- Process can destroy OS or other processes
- No privacy

## ■ Cannot move address space after it has been placed

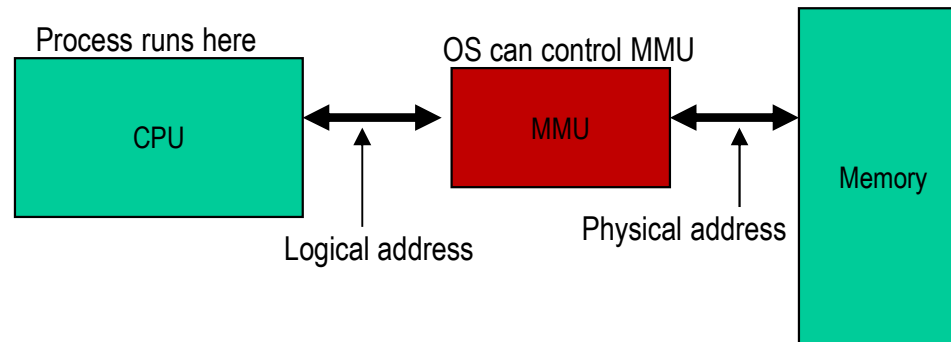
- May not be able to allocate new process

### **Fragmentation:**

If a new process arrives requiring a large block of memory, but available free spaces are fragmented (too small individually), the system may not be able to allocate new process.

### 3) Dynamic Relocation

- **Goal: Protect processes from one another**
- **Requires hardware support**
  - Memory Management Unit (MMU)
- **MMU dynamically changes process address at every memory reference**
  - Process generates **logical** or **virtual** addresses (in their address space)
  - Memory hardware uses **physical** or **real** addresses



# Hardware Support for Dynamic Relocation

## ■ Two operating modes

- Privileged (protected, kernel) mode: OS runs
  - When enter OS (trap, system calls, interrupts, exceptions)
  - Allows certain instructions to be executed
    - Can manipulate contents of MMU
  - **Allows OS to access all of physical memory**
- User mode: User processes run
  - **Perform translation of logical address to physical address**

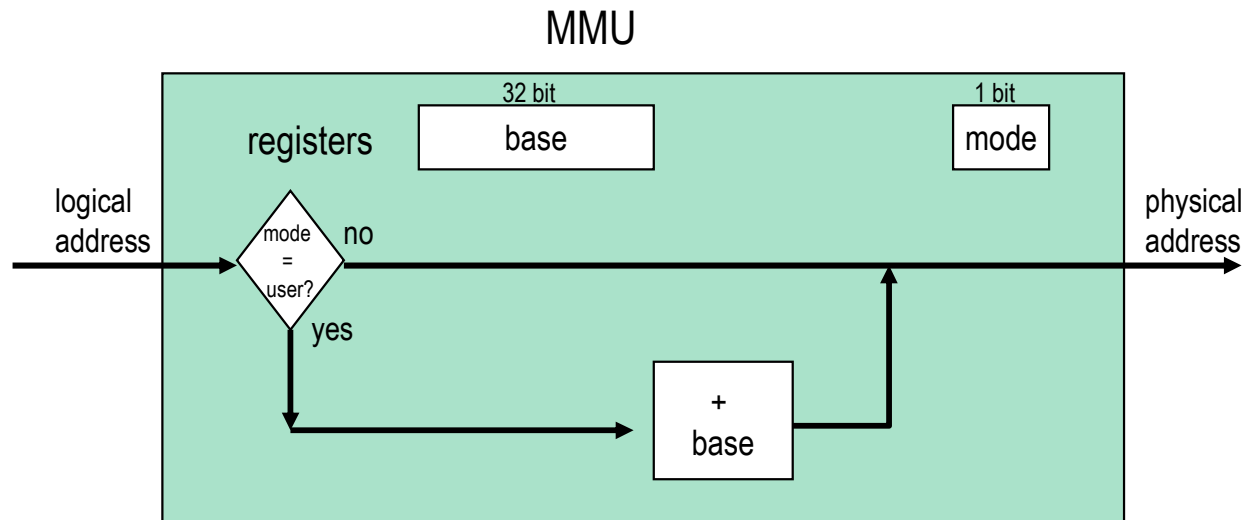
## ■ Minimal MMU contains base register for translation

- base: start location for address space



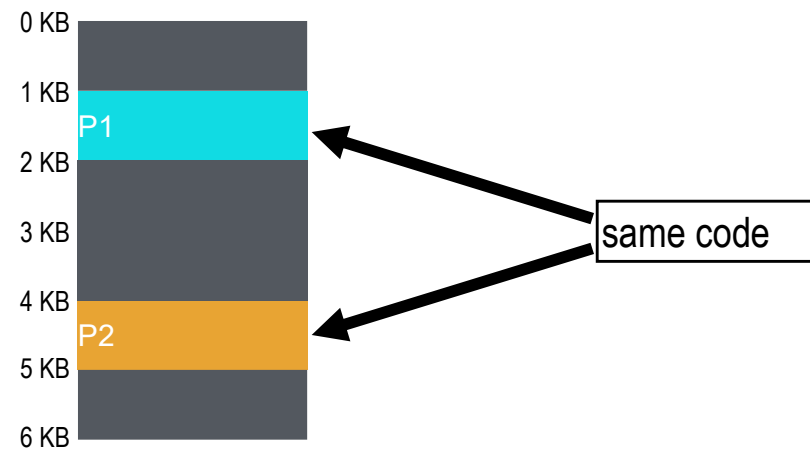
# Dynamic Relocation: BASE REG

- Translation on every memory access of user process
  - MMU adds base register to logical address to form physical address

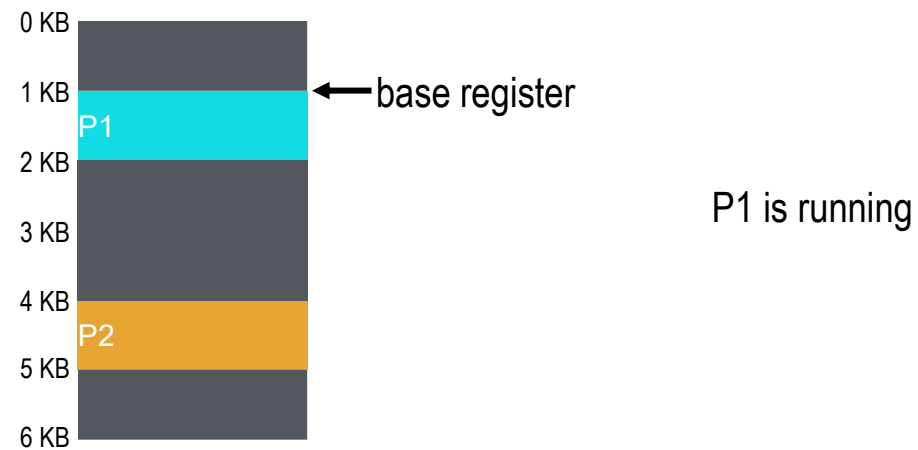


- Idea: translate virtual addresses to physical by adding a fixed offset each time.
  - Store offset in base register
  - Each process has different value in base register

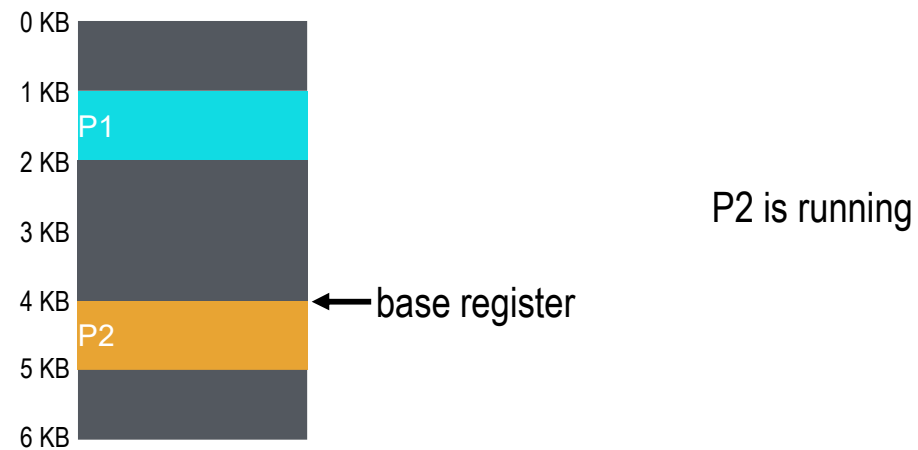
# VISUAL Example of DYNAMIC RELOCATION: BASE REGISTER



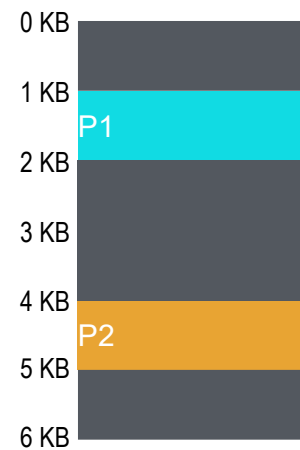
# VISUAL Example of DYNAMIC RELOCATION: BASE REGISTER



# VISUAL Example of DYNAMIC RELOCATION: BASE REGISTER



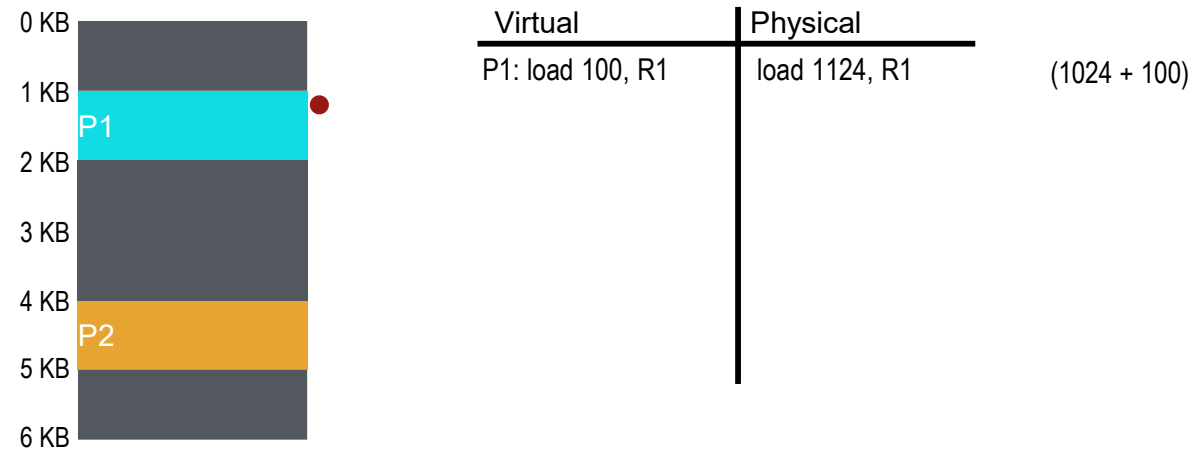
# VISUAL Example of DYNAMIC RELOCATION: BASE REGISTER



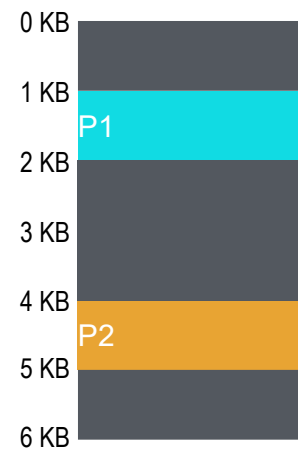
(Decimal notation)

| Virtual          | Physical |
|------------------|----------|
| P1: load 100, R1 |          |

# VISUAL Example of DYNAMIC RELOCATION: BASE REGISTER

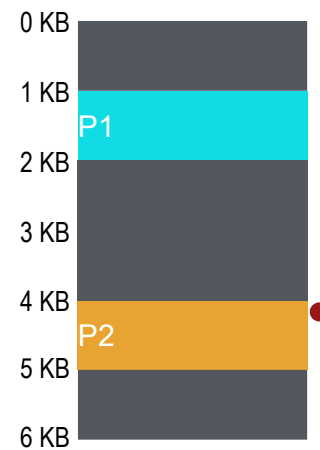


# VISUAL Example of DYNAMIC RELOCATION: BASE REGISTER



| Virtual          | Physical      |
|------------------|---------------|
| P1: load 100, R1 | load 1124, R1 |
| P2: load 100, R1 |               |

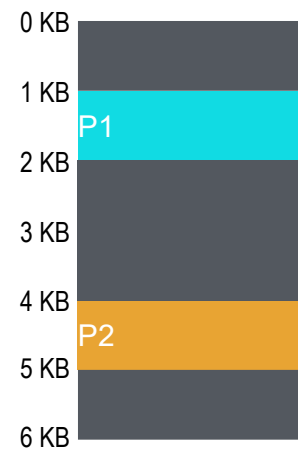
# VISUAL Example of DYNAMIC RELOCATION: BASE REGISTER



| Virtual          | Physical      |              |
|------------------|---------------|--------------|
| P1: load 100, R1 | load 1124, R1 |              |
| P2: load 100, R1 | load 4196, R1 | (4096 + 100) |

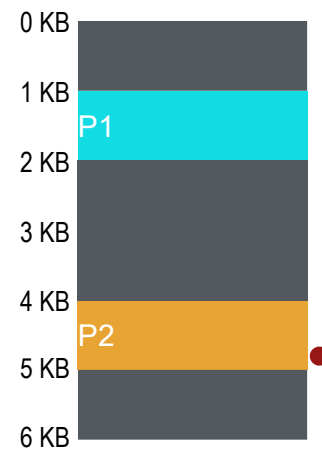


# VISUAL Example of DYNAMIC RELOCATION: BASE REGISTER



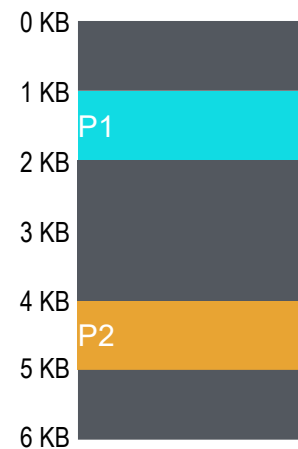
| Virtual           | Physical      |
|-------------------|---------------|
| P1: load 100, R1  | load 1124, R1 |
| P2: load 100, R1  | load 4196, R1 |
| P2: load 1000, R1 |               |

# VISUAL Example of DYNAMIC RELOCATION: BASE REGISTER



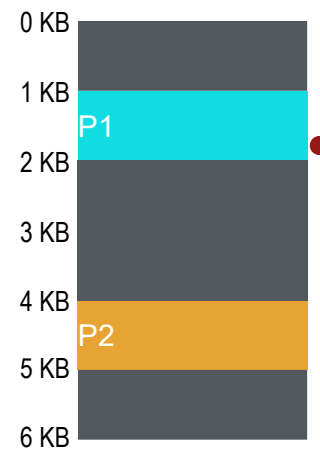
| Virtual           | Physical      |
|-------------------|---------------|
| P1: load 100, R1  | load 1124, R1 |
| P2: load 100, R1  | load 4196, R1 |
| P2: load 1000, R1 | load 5196, R1 |

# VISUAL Example of DYNAMIC RELOCATION: BASE REGISTER



| Virtual           | Physical      |
|-------------------|---------------|
| P1: load 100, R1  | load 1124, R1 |
| P2: load 100, R1  | load 4196, R1 |
| P2: load 1000, R1 | load 5196, R1 |
| P1: load 1000, R1 |               |

# VISUAL Example of DYNAMIC RELOCATION: BASE REGISTER



| Virtual           | Physical      |
|-------------------|---------------|
| P1: load 100, R1  | load 1124, R1 |
| P2: load 100, R1  | load 4196, R1 |
| P2: load 1000, R1 | load 5196, R1 |
| P1: load 1000, R1 | load 2024, R1 |

# Quiz: Who Controls the Base Register?

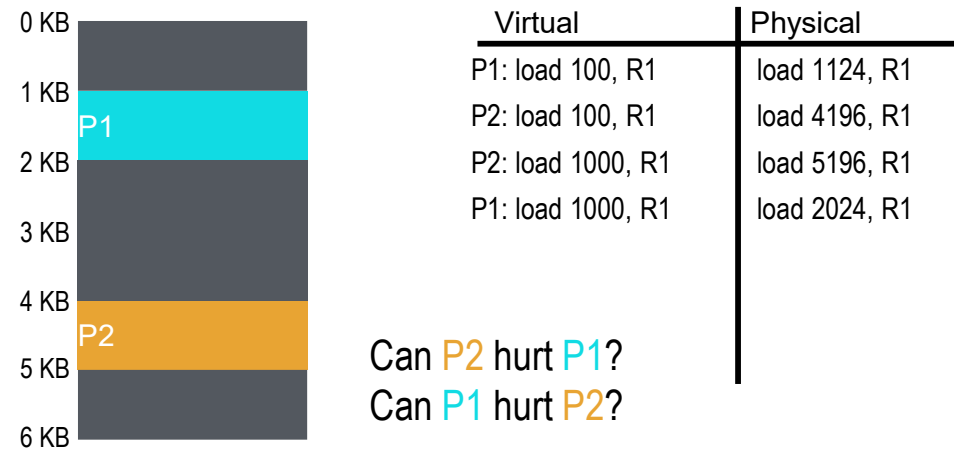
- What entity should **do translation** of addresses with base register?

(1) process, (2) OS, or (3) HW

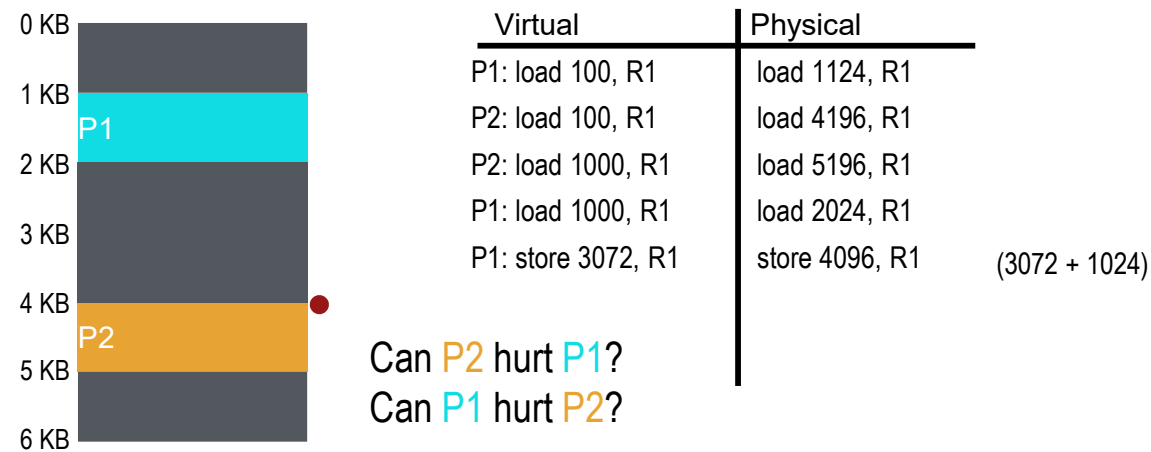
- What entity should **modify** the base register?

(1) process, (2) OS, or (3) HW

# How well does DYNAMIC RELOCATION do with BASE REGISTER for PROTECTION?



# How well does DYNAMIC RELOCATION do with BASE REGISTER for PROTECTION?



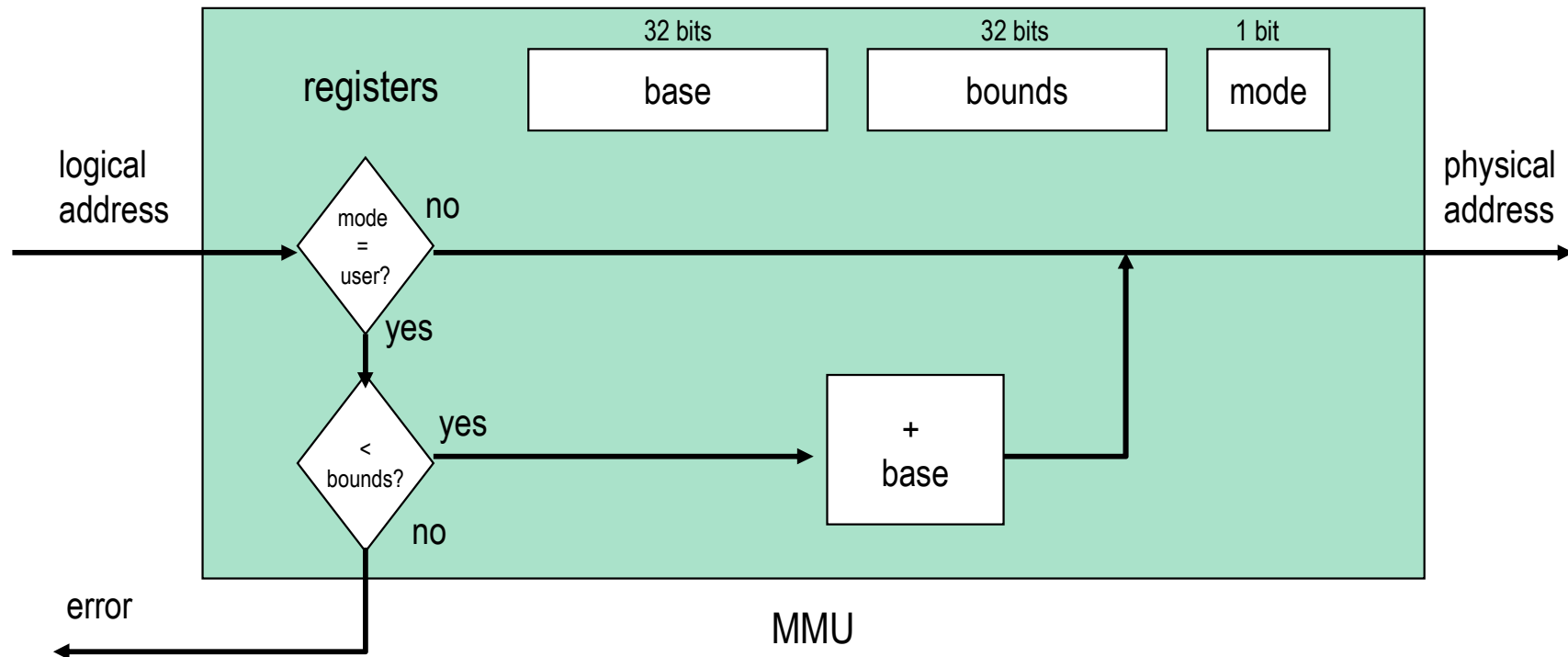
## 4) Dynamic with Base+Bounds

- **Idea: limit the address space with a bounds register**
  - **Base register: smallest** physical addr (or starting location)
  - **Bounds register: size** of this process's virtual address space
- **Sometimes defined as largest physical address (base + size)**
- **OS kills process if process loads/stores beyond bounds**

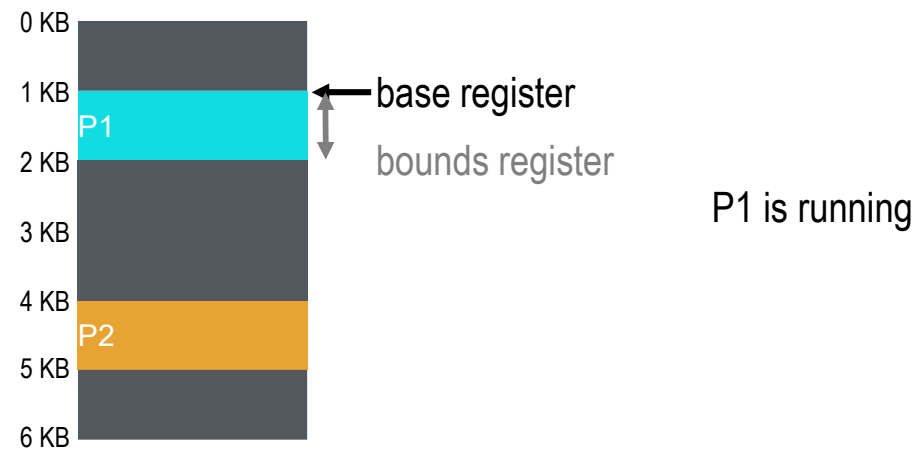


# Implementation of BASE+BOUNDS

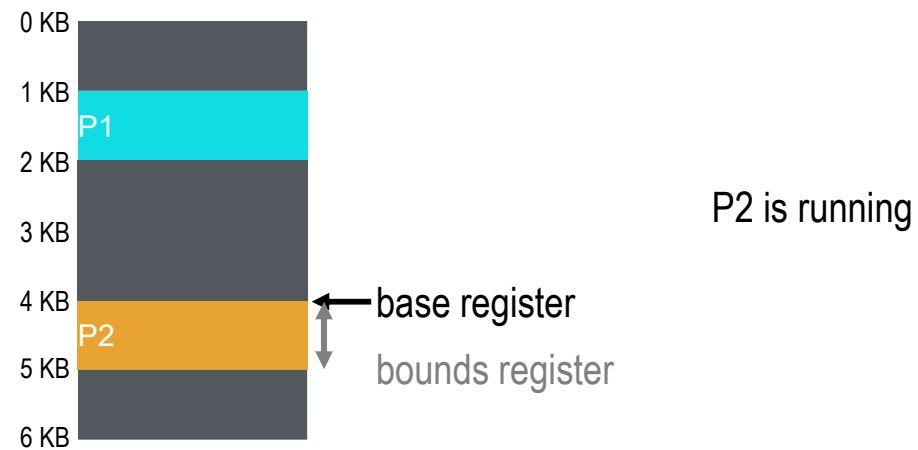
- **Translation on every memory access of user process**
  - MMU compares logical address to bounds register
    - if logical address is greater, then generate error
  - MMU adds base register to logical address to form physical address



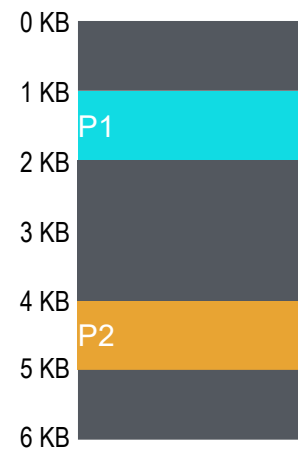
# VISUAL Example: BASE+BOUNDS REGISTERS



# VISUAL Example: BASE+BOUNDS REGISTERS

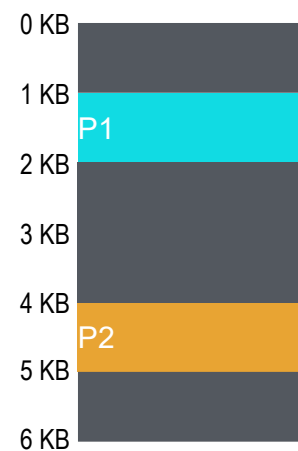


# VISUAL Example: BASE+BOUNDS REGISTERS



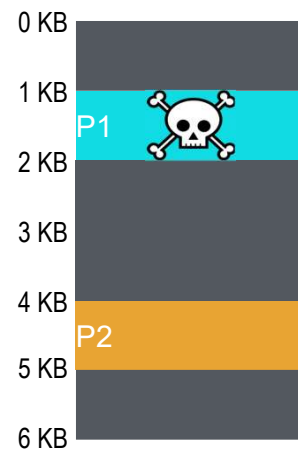
| Virtual            | Physical      |
|--------------------|---------------|
| P1: load 100, R1   | load 1124, R1 |
| P2: load 100, R1   | load 4196, R1 |
| P2: load 1000, R1  | load 5196, R1 |
| P1: load 100, R1   | load 2024, R1 |
| P1: store 3072, R1 |               |

Can P1 hurt P2?



| Virtual            | Physical      |
|--------------------|---------------|
| P1: load 100, R1   | load 1124, R1 |
| P2: load 100, R1   | load 4196, R1 |
| P2: load 1000, R1  | load 5196, R1 |
| P1: load 100, R1   | load 2024, R1 |
| P1: store 3072, R1 | interrupt OS! |

Can P1 hurt P2? 3072 > 1024



| Virtual            | Physical      |
|--------------------|---------------|
| P1: load 100, R1   | load 1124, R1 |
| P2: load 100, R1   | load 4196, R1 |
| P2: load 1000, R1  | load 5196, R1 |
| P1: load 100, R1   | load 2024, R1 |
| P1: store 3072, R1 | interrupt OS! |

Can P1 hurt P2?

# Managing Processes with Base and Bounds

## ■ Context-switch

- Add base and bounds registers to PCB
- Steps
  - Change to privileged mode
  - Save base and bounds registers of old process
  - Load base and bounds registers of new process
  - Change to user mode and jump to new process

## ■ What if don't change base and bounds registers when switch?

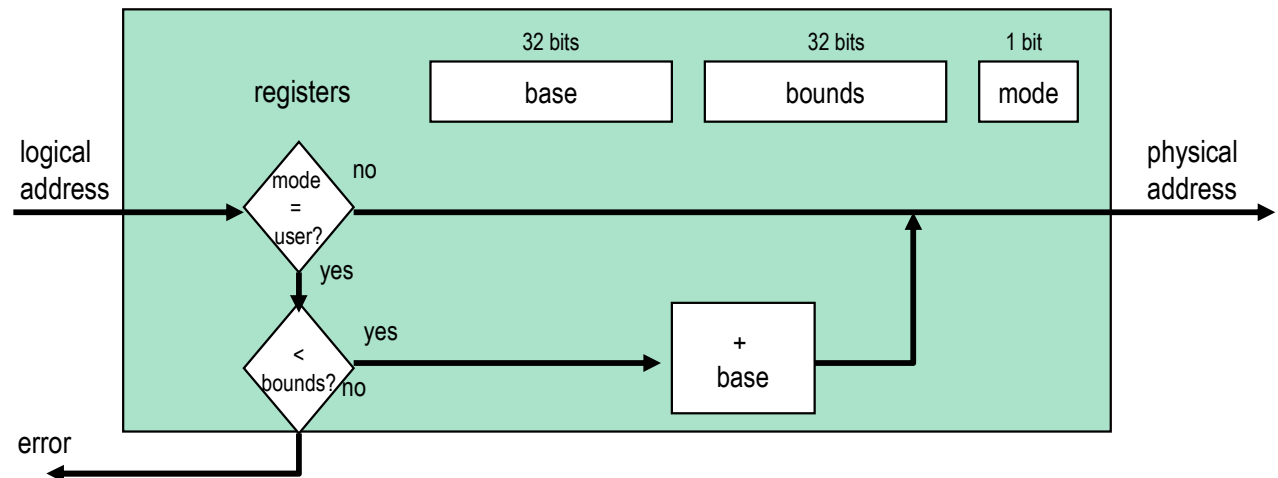
## ■ Protection requirement

- User process cannot change base and bounds registers
- User process cannot change to privileged mode

# Base and Bounds Advantages

## ■ Advantages

- Provides protection (both read and write) across address spaces
- Supports dynamic relocation
  - Can place process at different locations initially and also move address spaces
- Simple, inexpensive implementation
  - Few registers, little logic in MMU
- Fast
  - Add and compare in parallel

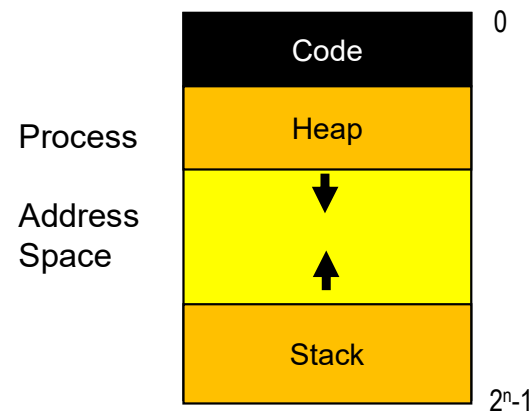




# Base and Bounds DISADVANTAGES

## ■ Disadvantages

- Each process must be **allocated contiguously** in physical memory
  - Must allocate memory that may not be used by process
- No partial sharing: Cannot share limited parts of address space



# Address Space and Translation: Summary

## ■ 1. Address Space: Process-Level Abstraction

- OS provides each process the *illusion* of private memory.
- Logical (virtual) addresses → translated to physical addresses.
- Components: code, static data, heap, stack.

## ■ 2. Why Virtualize Memory?

- Protection: processes cannot access each other or kernel.
- Transparency: processes don't know memory is shared.
- Efficiency: avoid fragmentation & use memory on demand.
- Sharing: allow shared libraries/pages.

## ■ 3. Mechanisms for Translation

- (1) Base Register (Dynamic Relocation)
  - $\text{Physical} = \text{Virtual} + \text{Base}$
- (2) Base + Bounds
  - Adds bounds check for protection.
  - Still requires contiguous memory; no sharing granularity.